PROJECT 7

Disaster Tweet Classification

Using NLP & Machine Learning

A complete end-to-end machine learning project from data preparation and EDA to model evaluation, serialization, web application and public deployment.

Submitted By	Satish Vonteddu
Program	Data Analytics Internship
Organization	NextHikes IT Solutions
Batch	L-110226-W2PM-B15
Submission Date	15 October 2026

GitHub: https://github.com/vsatish5621/Disaster-Tweet-Classification

Live Application: https://disaster-tweet-classification-baylfnxfscsp4xt4dmffxa.streamlit.app/

1. Executive Summary

This project builds an NLP-based classifier that predicts whether a tweet represents a real disaster-related message or a non-disaster message. The workflow covers text cleaning, tokenization and lemmatization, exploratory analysis, TF-IDF feature engineering, additional tweet-level features, model comparison, cross-validation, hyperparameter tuning, evaluation and deployment.

Final selected model: Tuned Multinomial Naive Bayes (alpha = 0.5). It achieved 81.55% accuracy, 83.48% precision, 71.10% recall and 76.80% F1 on the held-out test set.

The model was serialized and integrated into a user-facing application. A Flask version was tested locally and the final Streamlit version was deployed publicly through Streamlit Community Cloud.

Important data note: The project brief referenced 10,000 hand-classified tweets, while the supplied CSV contains 7,613 rows. The analysis and reported results use the supplied 7,613-row dataset.

2. Problem Statement & Objectives

Problem: Disaster-related keywords can appear in ordinary or metaphorical messages. A useful classifier therefore needs to consider the overall tweet text rather than relying only on keyword matching.

Objectives

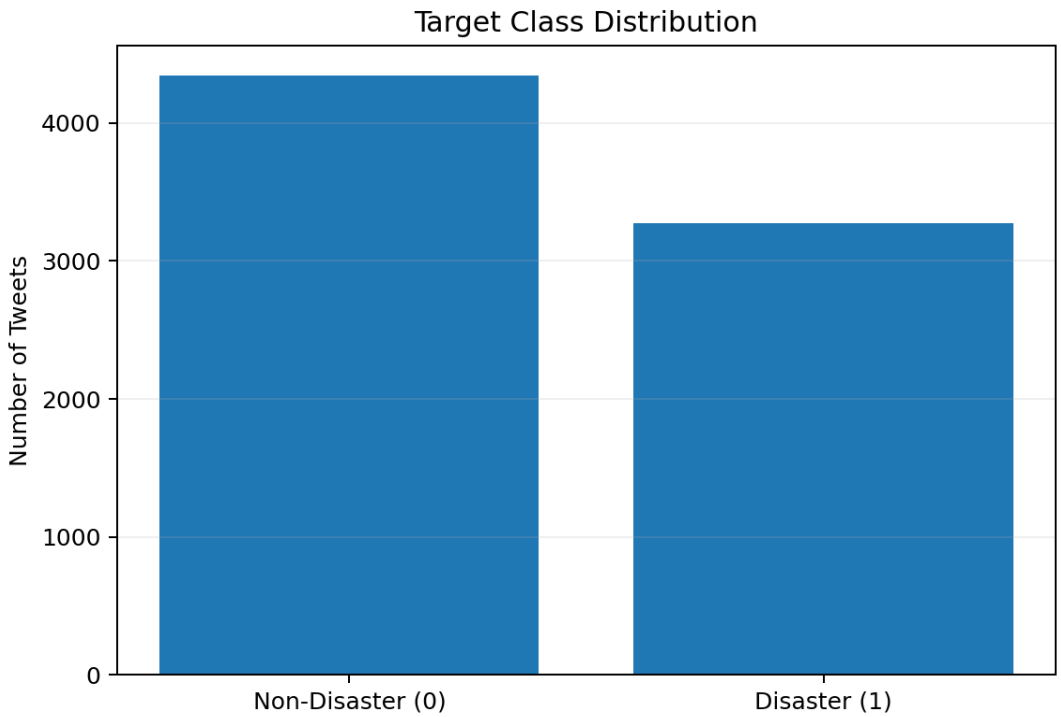
- Clean and prepare tweet text for NLP modeling.
- Explore class distribution, frequent terms and sentiment.
- Convert text into numerical features using TF-IDF.
- Add tweet length, hashtag and mention indicators.
- Train and compare multiple machine-learning models.
- Tune the strongest candidate and evaluate it with standard classification metrics.
- Serialize the model and vectorizer for inference.
- Provide a simple web interface and deploy it publicly.

3. Dataset & Exploratory Data Analysis

Dataset: twitter_disaster.csv

Shape: 7,613 rows × 5 columns — id, keyword, location, text, target.

Missing values: keyword = 61; location = 2,534. The target and text fields were used for modeling; missing metadata did not prevent the NLP workflow.



Class distribution: 4,342 non-disaster tweets (57.03%) and 3,271 disaster tweets (42.97%).

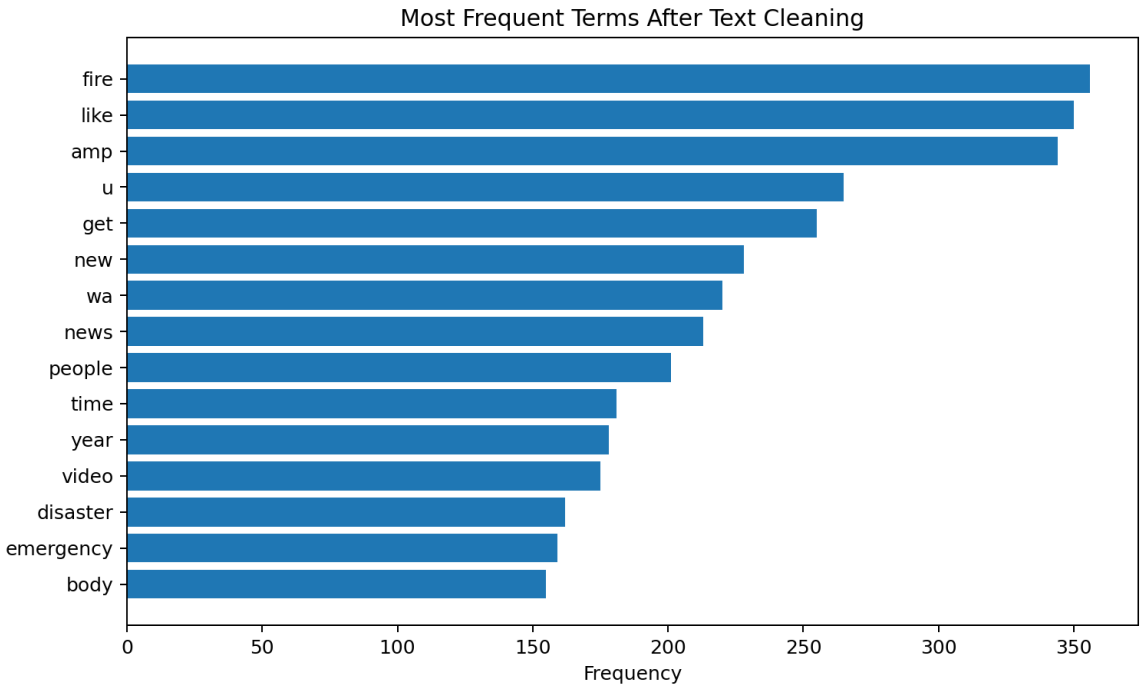
4. Text Cleaning & Feature Engineering

The text pipeline was designed to reduce noise while preserving useful linguistic information.

- Lowercase conversion.
- Removal of URLs and HTML fragments.
- Removal of newlines, punctuation and special characters.
- Whitespace normalization.
- Tokenization.
- English stopword removal.
- WordNet lemmatization.
- Creation of tweet_len, has_hashtag and has_mention features.
- TF-IDF representation with max_features = 5,000.
- Final feature space = 5,003 features (5,000 TF-IDF + 3 additional features).

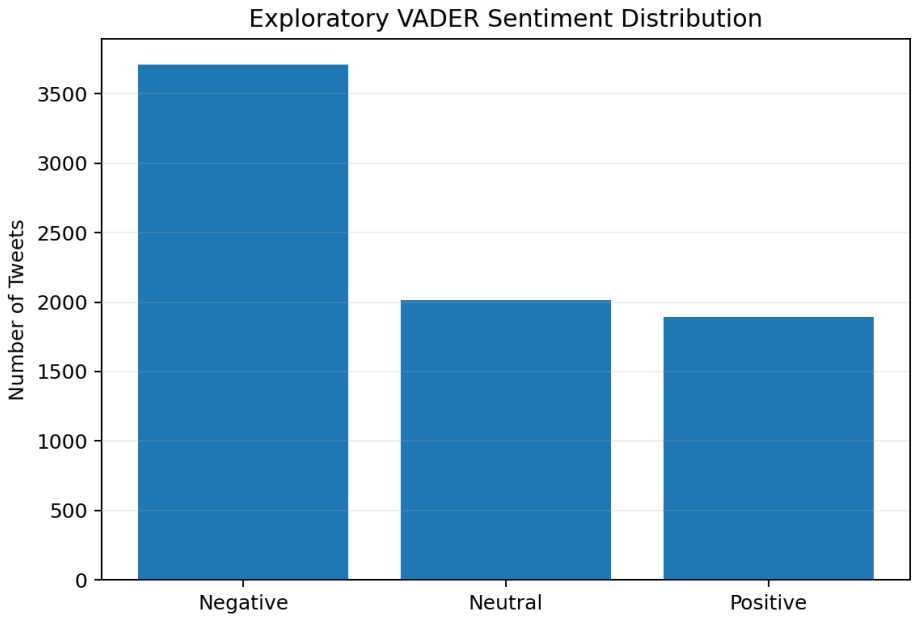
Train/test split: 80/20 stratified split with random_state = 42 — 6,090 training rows and 1,523 test rows.

5. Frequent Terms



The frequency analysis was performed after text cleaning. Common terms included fire (356), like (350), amp (344), u (265), get (255), new (228), wa (220), news (213), people (201), time (181), year (178), video (175), disaster (162), emergency (159), body (155), day (151), home (144), police (143) and building (141).

6. Exploratory Sentiment Analysis

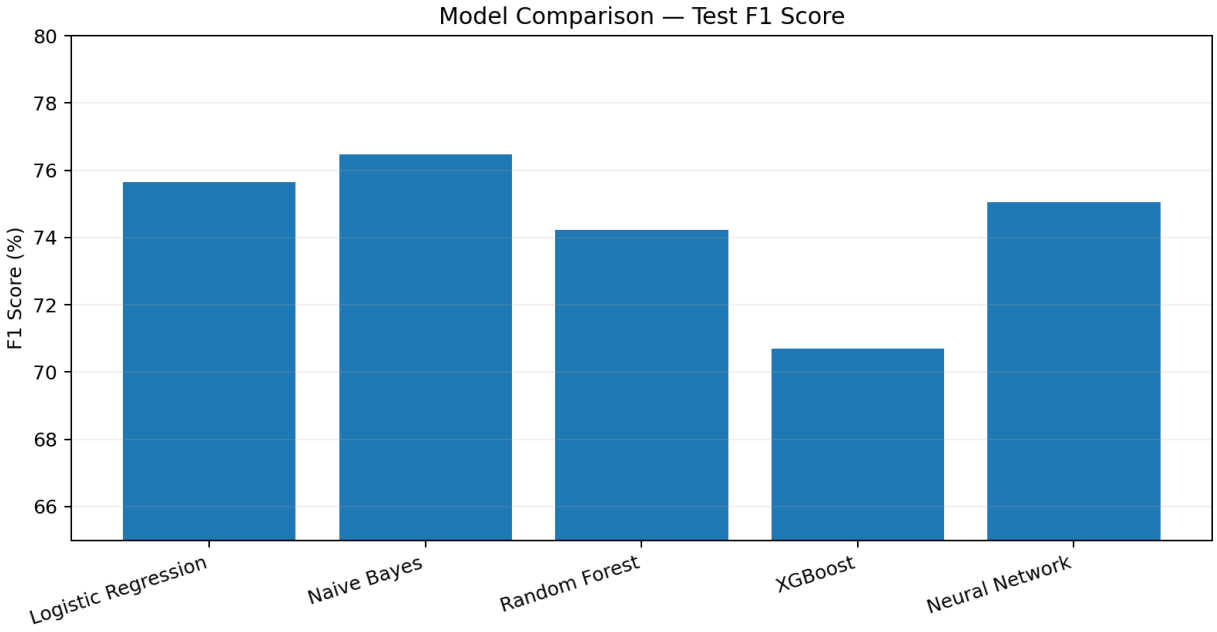


VADER sentiment was used as an exploratory analysis only. The dataset contained 3,708 negative, 2,012 neutral and 1,893 positive tweets. Sentiment was not added to the final selected model.

7. Model Development & Comparison

Five candidate models were trained on the engineered features. The test-set results are summarized below.

Model	Accuracy	Precision	Recall	F1
Logistic Regression	80.56%	81.85%	70.34%	75.66%
Naive Bayes	81.81%	86.04%	68.81%	76.47%
Random Forest	79.97%	82.99%	67.13%	74.22%
XGBoost	77.74%	81.31%	62.54%	70.70%
Neural Network	80.89%	85.38%	66.97%	75.06%



The initial best F1 score among the tested models was Logistic Regression at 75.66%, closely followed by Naive Bayes at 76.47%.

8. Cross-Validation & Hyperparameter Tuning

5-fold cross-validation F1: Logistic Regression mean = 0.7527 (std = 0.0082); Naive Bayes mean = 0.7394 (std = 0.0144).

Logistic Regression tuning: tested C = [0.1, 1, 10] and solvers liblinear/lbfgs. Best configuration was C = 1, solver = lbfgs; test F1 remained 75.66%.

Naive Bayes tuning: tested alpha = [0.01, 0.1, 0.5, 1, 2]. Best alpha = 0.5.

The tuned Multinomial Naive Bayes model was selected for the final application.

9. Final Model Performance

Tuned Multinomial Naive Bayes — alpha = 0.5

Metric	Test Result
Accuracy	81.55%

Metric	Test Result
Precision	83.48%
Recall	71.10%
F1 Score	76.80%
ROC-AUC	87.27%
Average Precision	87.28%

Confusion matrix: TN = 777, FP = 92, FN = 189, TP = 465.

10. Confusion Matrix & Error Analysis

Actual vs Predicted

	Predicted 0	Predicted 1
Actual 0	777 (TN)	92 (FP)
Actual 1	189 (FN)	465 (TP)

- True negatives: 777 non-disaster tweets correctly identified.
- False positives: 92 non-disaster tweets predicted as disaster.
- False negatives: 189 disaster tweets missed by the classifier.
- True positives: 465 disaster tweets correctly identified.

11. Overfitting Check

Training F1 = 0.8382 and test F1 = 0.7680, giving a gap of 0.0702 (7.02 percentage points). This indicates a moderate training-test performance gap. The model is useful but not perfectly generalizable.

12. Robustness & Practical Validation

The serialized model was reloaded successfully and used for inference. A Colab validation example — “There is a massive earthquake and buildings are collapsing” — was classified as Disaster with 83.95% probability. The public application was also tested with both disaster and non-disaster examples.

13. Model Serialization

- disaster_tweet_model.pkl — final trained MultinomialNB model.
- tfidf_vectorizer.pkl — fitted TF-IDF vectorizer.
- Both artifacts were reloaded successfully for prediction.

14. Application & Deployment Architecture

Flow: User Tweet → Text Preprocessing → TF-IDF + Tweet Features → Serialized Model → Prediction + Confidence → User Interface

Local application: Flask app (app.py + Templates/index.html) was tested locally.

Public application: Streamlit application (disaster_classifier_app.py) deployed on Streamlit Community Cloud.

Deployment Python version: 3.13.

15. Live Application Validation

Two public deployment tests were completed:

- Disaster example involving flooding/evacuation → Disaster Tweet, 93.14% confidence.
- Metaphorical cooking example (“disaster trying to cook dinner...”) → Non-Disaster Tweet, 79.69% confidence.

Live URL: https://disaster-tweet-classification-baylfnxfscsp4xt4dmffxa.streamlit.app/

GitHub: https://github.com/vsatish5621/Disaster-Tweet-Classification

16. Repository Structure

Key files committed to GitHub:

- README.md — complete project documentation and submission links.
- disaster_classifier_app.py — Streamlit deployment application.
- app.py — Flask application.
- Templates/index.html — Flask UI.
- disaster_tweet_model.pkl — trained model.
- tfidf_vectorizer.pkl — TF-IDF vectorizer.
- requirements.txt — Python dependencies.
- .gitignore, .python-version, LICENSE — project configuration and repository files.

17. Important Technical Limitation

For this project run, the TF-IDF vectorizer was fitted before the train/test split. This introduces a mild form of data leakage because vocabulary statistics were exposed to the full dataset. The reported test metrics should therefore be interpreted as the results of this completed project run, not as a strictly leakage-free benchmark.

Recommended improvement: use a scikit-learn Pipeline so TF-IDF fitting occurs only on the training folds during cross-validation and model training.

18. Future Improvements

- Use a leakage-free Pipeline and nested/stratified validation.
- Experiment with Word2Vec/GloVe or transformer embeddings.
- Tune class weighting and decision thresholds to improve disaster recall.
- Add more robust error analysis for ambiguous/metaphorical tweets.
- Add richer UI visualizations and prediction history.
- Evaluate on a separate external/unseen disaster-tweet dataset.

19. Requirement Coverage

The completed project covers the requested workflow: EDA and preparation; text cleaning and tokenization; class labeling; train/test split; word-frequency analysis; TF-IDF; exploratory sentiment; additional tweet features; multiple candidate models; cross-validation; hyperparameter tuning; accuracy/precision/recall/F1; confusion matrix; ROC-AUC and precision-recall evaluation; model comparison; serialization; Flask local interface; Streamlit public deployment; GitHub repository; and live validation.

20. Final Conclusion

The project demonstrates a complete practical NLP classification workflow. The tuned Multinomial Naive Bayes model provides a solid baseline with 81.55% test accuracy and 76.80% F1, while the deployed application makes the trained model accessible through a simple interface. The documented leakage limitation provides a clear path for strengthening the next iteration.

Submission Links

GitHub Repository: https://github.com/vsatish5621/Disaster-Tweet-Classification

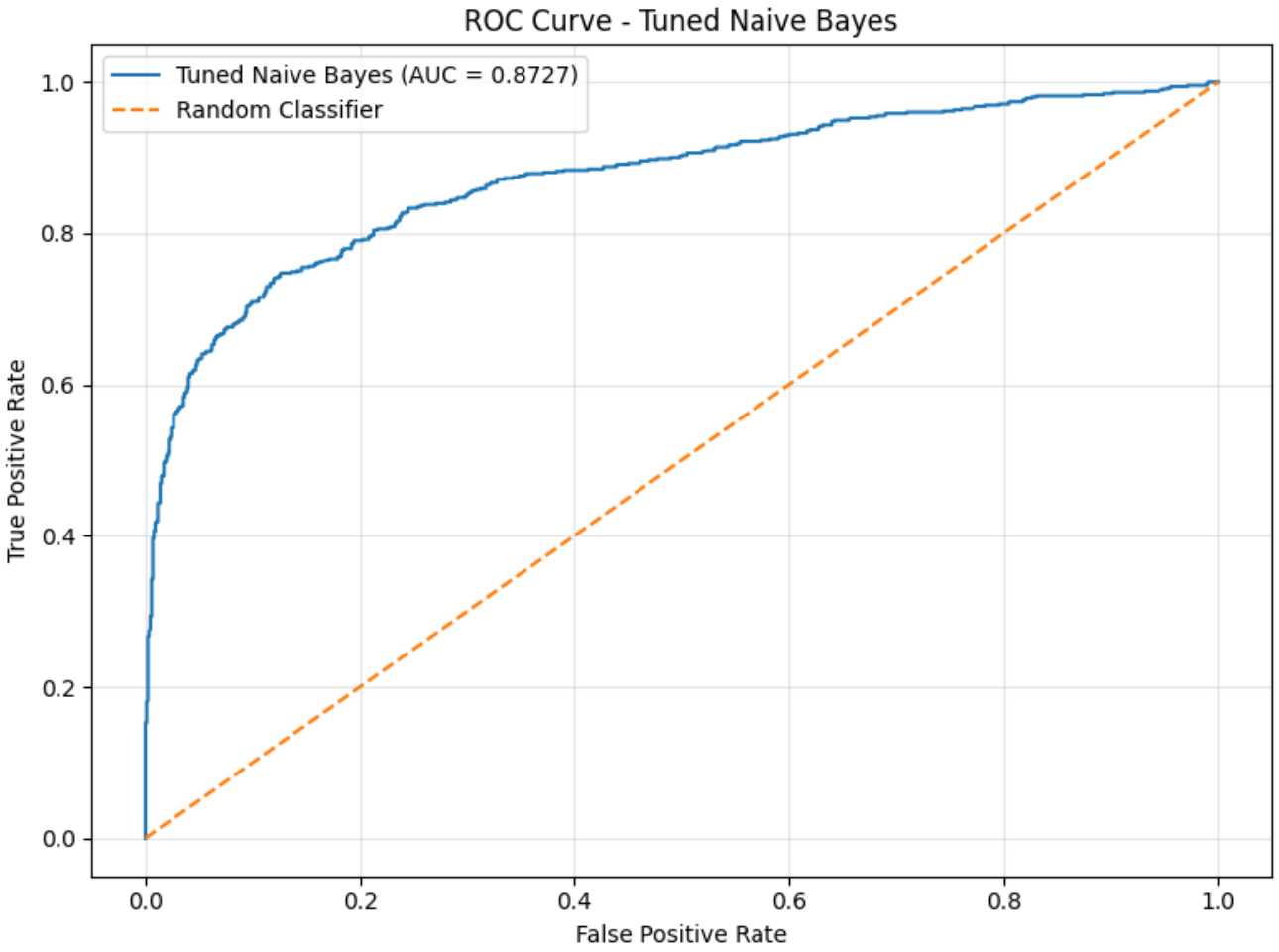
Live Streamlit Application: https://disaster-tweet-classification-baylfnxfscsp4xt4dmffxa.streamlit.app/

Submitted by Satish Vonteddu | Data Analytics Internship | NextHikes IT Solutions | Batch L-110226-W2PM-B15 | 15 October 2026

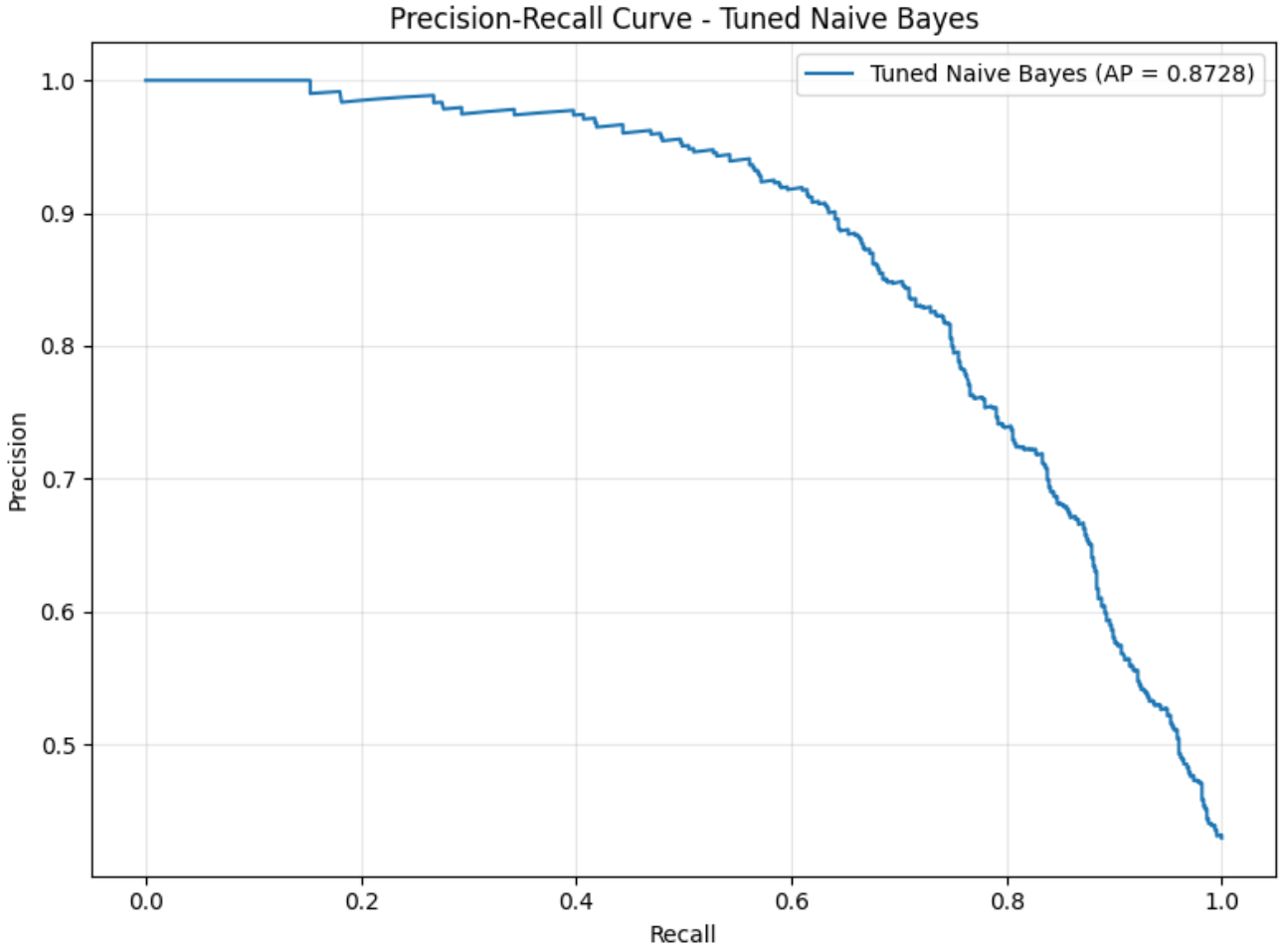
Project 7 – Final Model Evaluation Curves

The original Project 7 notebook was checked and contains the actual ROC and Precision–Recall curve calculations and rendered plots for the tuned Multinomial Naive Bayes model. These figures are reproduced here without fabricating or estimating threshold values.

ROC Curve – Tuned Naive Bayes (ROC-AUC = 0.8727)



Precision–Recall Curve – Tuned Naive Bayes (Average Precision = 0.8728)


Precision-Recall Curve - Tuned Naive Bayes
Tuned Naive Bayes (AP = 0.8728)
Precision
Recall
1.0
0.9
0.8
0.7
0.6
0.5
0.0
0.2
0.4
0.6
0.8
1.0